0005. Longest Palindromic Substring

Medium · 1-D Dynamic Programming
leetcode.com/problems/longest-palindromic-substring

Problem

- Return the longest palindromic substring of s.

Intuition

- Every palindrome has a centre: a character (odd length) or a gap between two characters (even length).
- Expand outward from each of the 2n-1 centres while the ends match.

Brute Force

- Check all O(n^2) substrings for O(n) each: O(n^3). A DP table is O(n^2) time and space.

Optimal Approach

- For each i: expand(i, i) and expand(i, i+1).
- Expand returns the palindrome length; update the best start/length.
- (Manacher's algorithm is O(n), mention as follow-up.)

Worked Example

```
"babad".

centre 0 'b'   -> "b"
centre 1 'a'   -> "bab" (len 3)
centre 2 'b'   -> "aba" (len 3)
even centres   -> nothing longer
answer "bab"
```

Java Solution

```java
public class LongestPalindromicSubstring {
    public String longestPalindrome(String s) {
        int start = 0, best = 0;
        for (int i = 0; i < s.length(); i++) {
            int len = Math.max(expand(s, i, i), expand(s, i, i + 1));
            if (len > best) { best = len; start = i - (len - 1) / 2; }
        }
        return s.substring(start, start + best);
    }

    private int expand(String s, int l, int r) {
        while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
        return r - l - 1;
    }
}
```

Complexity

- Time: O(n^2). Space: O(1).

Pitfalls

- Do not forget even-length centres ("cbbd" → "bb").